reduce/chain

An AgentMPI collective scaling analysis

Abstract

The chain reduction is the serial left fold: rank $p-1$ starts with its own contribution and each rank in turn combines the incoming accumulator with its own and passes it down, so the result lands at rank 0 having been through $p-1$ operator applications in strict rank order. It is the reference semantics every other reduce algorithm is tested against, and the only algorithm the runtime will use for an operator declared non-associative. Across 21 measured sizes from $p=2$ to $p=32$ it logged exactly $p-1$ messages, reported that count exactly, recorded $p-1$ rounds, and recorded a `fold_depth` of exactly $p-1$ — all four agreeing with the closed form at 21 of 21, with no remainder path to get wrong and no red crosses. The trace shows the fold depth incrementing one hop at a time: at $p=8$ the successive `coll.reduce` records read $0, 1, 2, 3, 4, 5, 6, 7$ down the line from rank 7 to rank 0. This is the family that prices depth for a reduction, and the price is steep: maximum per-rank blocking rises from 10.5 ms at $p=2$ to 2,484.9 ms at $p=32$, twenty-four times the binomial tree's 101.8 ms and fifty times the flat reduction's 49.2 ms at the same size. The single most important thing to take away is the asymmetry with the mirrored `bcast/chain`, which pays the same $p-1$ hops: the broadcast's $p-1$ messages carry exactly *one* distinct content digest at every size, while this reduction's carry $p-1$ *distinct* digests — a new artifact per hop — even though every rank contributed the identical value 1. A chain broadcast is slow and faithful; a chain reduction is slow and rewrites its payload thirty-one times. I also report a minor accounting defect: the collective under-reports its own token volume by a fixed 15 tokens per message.

1 What this family is

The `reduce` collective under the `chain` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- [ok] All 21 measured sizes logged exactly the number of messages the closed form predicts.
- [note] Wall times span 0.012–2.494 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

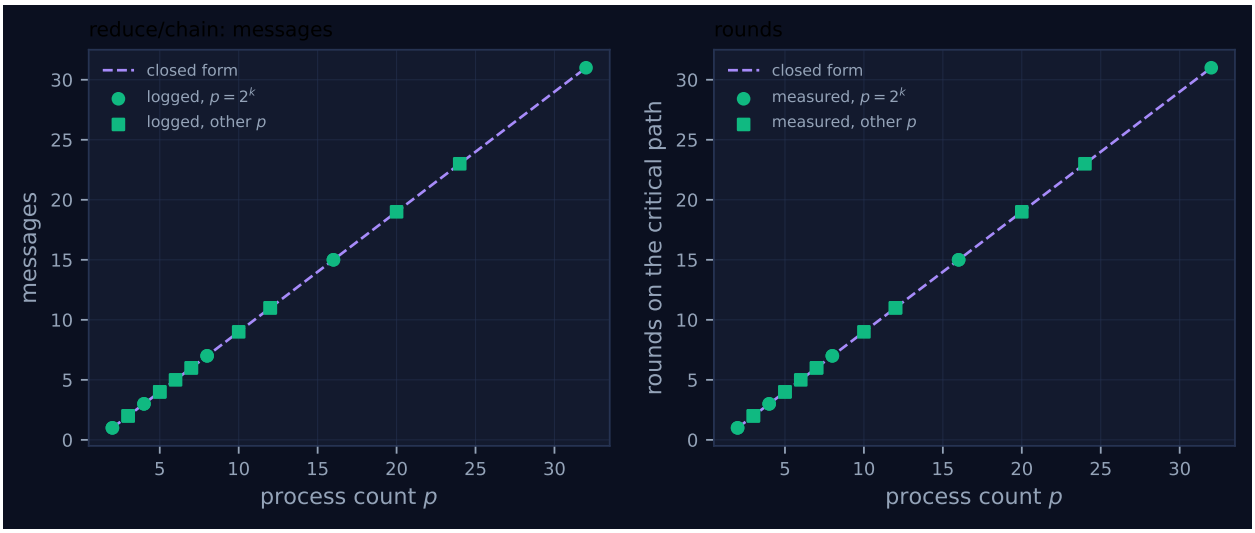

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	1	1	1	1	1	0.013	✓	✓
2	`mb-smoke`	1	1	1	1	1	0.012	✓	✓
3	`mb-free`	2	2	2	2	2	0.026	✓	✓
3	`mb-smoke`	2	2	2	2	2	0.029	✓	✓
4	`mb-free`	3	3	3	3	3	0.026	✓	✓
4	`mb-smoke`	3	3	3	3	3	0.030	✓	✓
5	`mb-free`	4	4	4	4	4	0.046	✓	✓
5	`mb-smoke`	4	4	4	4	4	0.066	✓	✓
6	`mb-free`	5	5	5	5	5	0.070	✓	✓
7	`mb-free`	6	6	6	6	6	0.097	✓	✓
7	`mb-smoke`	6	6	6	6	6	0.046	✓	✓
8	`mb-free`	7	7	7	7	7	0.094	✓	✓
8	`mb-smoke`	7	7	7	7	7	0.241	✓	✓
10	`mb-free`	9	9	9	9	9	0.186	✓	✓
12	`mb-free`	11	11	11	11	11	0.199	✓	✓
12	`mb-smoke`	11	11	11	11	11	0.330	✓	✓
16	`mb-free`	15	15	15	15	15	0.432	✓	✓
16	`mb-smoke`	15	15	15	15	15	0.433	✓	✓
20	`mb-free`	19	19	19	19	19	1.246	✓	✓
24	`mb-free`	23	23	23	23	23	1.747	✓	✓
32	`mb-free`	31	31	31	31	31	2.494	✓	✓

4 How the algorithm scales

The fold runs from the far end inward, and the direction is deliberate. Rank $p-1$ has no predecessor, so it starts with `acc = payload`, `weight = 1`, `depth = 0`. Every other rank receives from $r+1$, takes the incoming `depth` and `weight`, increments both, and combines with its *own* payload as the

left operand:

```
acc = op.fn(payload, got["acc"], _reduce_ctx(comm, depth, weight))
```

so the accumulator at rank r is $\mathrm{op}(payload_r, \mathrm{op}(payload_{r+1}, \dots))$ and the evaluation order is exactly ascending rank order. That is the guarantee a non-commutative operator needs, and it is why the chain is the reference implementation: for an EXACT operator every other algorithm must reproduce this result, and the test suite checks that across nine sizes. The result lands at rank 0; if the root is elsewhere it takes one extra hop, which is what MPI implementations do for the same reason.

Each of the $p-1$ non-root ranks sends exactly once, so the traffic is $p-1$ messages and $(p-1)n$ tokens; the dependency chain is $p-1$ long, so both the round count and the fold depth are $p-1$. That is `FORMULAS[("reduce","chain")]`, which records $(p-1,\ p-1,\ (p-1)n,\ p-1)$, and this is the one reduce algorithm in which all four terms except the volume are the same integer. It is also the only algorithm in the reduce family for which the round count and the fold depth coincide with each other *and* with the application count, which makes it the natural control: any difference another algorithm shows against it is attributable to arrangement rather than to work.

The trace is a clean staircase. In `mb-free__coll-reduce-chain-8`, rank 7 sends at 6.20 ms with `fold_depth: 0`; rank 6 receives at 14.94, sends at 16.76 with `fold_depth: 1`; rank 5 at 27.80 and 29.21 with depth 2; rank 4 at 44.27 and 46.02 with depth 3; rank 3 at 47.49 and 48.67 with depth 4; rank 2 at 66.16 and 67.70 with depth 5; rank 1 at 72.16 and 74.12 with depth 6; and rank 0 receives at 93.67 and records `fold_depth: 7` with 92.1 ms of blocking. No two hops overlap, the gap between a reception and the corresponding forward is consistently 1.2–2.0 ms (constructing the envelope and applying `SUM`), and the gap between a forward and the next reception ranges from 1.4 to 19.5 ms. The depth increments by exactly one per hop at every size, so the measured fold depth is not merely equal to $p-1$ in aggregate — the whole sequence is right.

This is the one reduce family whose wall-clock numbers carry an argument, because they are large enough to escape the harness's noise floor. Maximum per-rank blocking runs 10.5 ms at $p=2$, 23.8 at $p=3$, 92.1 at $p=8$, 428.5 at $p=16$, 1,237.2 at $p=20$, 1,734.5 at $p=24$ and 2,484.9 at $p=32$. It is not linear in $p-1$, though. Dividing by the hop count gives 10.5 ms per hop at $p=2$, 13.2 at $p=8$, 28.6 at $p=16$, 65.1 at $p=20$, 75.4 at $p=24$ and 80.2 at $p=32$: a factor of six over the range, when nothing in the algorithm changes with p. The same rise appears in `bcast/chain` (10.4 to 47.8 ms per hop) and does not appear in the flat or binomial families, where far fewer ranks are blocked simultaneously. My reading is that it is contention: $p-1$ ranks sit polling the same SQLite fabric while exactly one hop of useful work happens at a time. That is a property of this harness, not of chain reduction, and it means the measured chain-versus-tree ratio overstates what the cost model predicts.

Figure 1 is accordingly the least eventful figure in the reduce set, and deliberately so: two identical straight lines through every marker, powers of two and other sizes alike, no staircase and no crosses, because for this algorithm the message count and the round count are the same integer. Its value is as a control — the binomial panel's staircase means something only because this panel shows what no depth reduction looks like.

The viewer screenshot for `mb-free__coll-reduce-chain-32` shows the shape unmistakably. The stats row reports a 2.5 s span, 31 messages, 0 agent calls, $0.000 and 0 failures. The timeline is a staircase running from high rank to low: rank 21's glyphs sit at roughly 0.27 s, and the marks march rightward as the rank index falls, reaching rank 2 at about 2.4 s and rank 0 at 2.5 s. Every lane holds one or two instantaneous ticks and no bars, and at any instant exactly one lane is active while

thirty-one wait. It is worth putting next to `mb-free__coll-bcast-chain-32`, whose staircase runs the other way — rank 0 first, rank 31 last. The two pictures are mirror images, which is the duality made literal, and the run-level metric that captures it is `coordination_span_share = 0.9962`: essentially the entire run is one collective serialising.

5 Powers of two and everything else

There is no remainder path. The arithmetic is $r + 1$ and $r - 1$ with no modular wrap and no branch on p; the only conditional is `if r == p - 1` for the initiator and `if r > 0` for the terminator. The code executed at $p = 7$ is the code executed at $p = 8$ with one fewer hop. This is the strongest possible answer to the powers-of-two question: not that the remainder path behaves, but that no such path exists.

The measurements agree. All eight non-power-of-two sizes logged $p - 1$ messages (2 at $p = 3$, 4 at $p = 5$, 5 at $p = 6$, 6 at $p = 7$, 9 at $p = 10$, 11 at $p = 12$, 19 at $p = 20$, 23 at $p = 24$), reported the same figure the fabric logged, and recorded rounds and fold depth both equal to $p - 1$. Both marker classes lie on the closed form in both panels of Figure 1, at every size.

A minor accounting defect: token volume is under-reported by a fixed envelope

Message counts agree everywhere; token volumes do not. Summing the `tokens` field of the `msg.send` events gives $16(p - 1)$ at every size — 112 at $p = 8$, 240 at $p = 16$, 496 at $p = 32$ — while the collective reports $p - 1$ as `tokens_sent`: 7, 15 and 31. The cause is a one-line mismatch: the algorithm transmits a three-field envelope, `{"acc": ..., "depth": ..., "weight": ...}`, and the tracker charges `tr.sent(_tok(comm, acc))`, the accumulator alone. Measuring `tokens.count` on the same envelope with string payloads from 1 to 8,200 tokens shows the difference is a fixed additive constant of 16 tokens, independent of payload size, so the under-report is 15 tokens per message here and $\approx 16(p - 1)$ in general.

The magnitude is bounded — about 2% at a realistic 1,000-token artifact, and large in relative terms here only because the payload is a single token — but the direction is the one the repository's own accounting test warns about in its docstring: "an under-reporting collective looks cheaper than it is, which is the one direction of error a cost model must never have." The reason it survived is that `test_self_reported_message_count_equals_the_traffic_actually_logged` compares `messages_sent` against the `msg.send` count and never compares `tokens_sent` against the logged `tokens`. `reduce/binomial` has the same defect at 19 tokens per message, its envelope carrying a fourth field; `reduce/flat` has none, because it sends the payload bare. I would call it a real if small defect, and the fix is to charge `_tok` of the envelope actually sent.

Eight sizes were measured twice, under `mb-free` and `mb-smoke`. Every count is identical at all eight. The run wall times differ by factors from 1.00 to 2.56 — 0.094 against 0.241 s at $p = 8$, and 0.432 against 0.433 s at $p = 16$ — which is the right summary of a measurement whose dominant term is scheduler behaviour under contention.

6 When to choose this algorithm

Chain reduction is the algorithm you use when you have no choice, and the sweep is fairly read as the case against using it when you do.

There are two situations in which it is the only legal option. The first is an operator declared `Associativity.NONE`: the runtime refuses every tree for such an operator and `reduce` defaults to `chain`, on the grounds that requesting a tree for a non-associative operator should be an error rather than a silent quality regression. The second is a non-commutative operator at a non-zero root, where `binomial` raises because virtual-rank order no longer coincides with communicator rank order; `chain` and `flat` remain available. Beyond legality, chain is the reference semantics — the serial left fold that `Op.fold` defines and that every other algorithm is tested against — so it is the algorithm to run when the question is what the answer *should* be.

Everywhere else the cost is prohibitive, and this is where the sweep's timings earn their keep. All three of `flat`, `binomial` and `chain` send $p-1$ messages and perform $p-1$ operator applications, so they differ only in arrangement, and chain's arrangement is maximally serial on both axes: $p-1$ rounds and $p-1$ successive applications, against $\lceil \log_2 p \rceil$ and $\lceil \log_2 p \rceil$ for binomial and 1 and $p-1$ for flat. In the agent-free sweep that shows as 2,484.9 ms of blocking at $p = 32$ against binomial's 101.8 and flat's 49.2 — factors of 24 and 50 — where the cost model predicts a chain-to-binomial ratio of $31/5 = 6.2$. The measured penalty is four times the modelled one because the per-hop cost grows with p; the 6.2 is the figure to quote for a real population.

The archived agent-executed runs at $p = 8$ confirm the ordering with a real operator and a real α. Root blocking was 1,377.0 s and 634.4 s under `chain`, against 197.6 and 251.6 under `flat`, 77.6 and 80.3 under `binomial`, and 53.1 and 51.8 under `kary`(k=4). Chain is slowest of the four in both campaigns, by factors of 8 to 17 over the binary tree and 12 to 26 over the 4-ary one, and it emitted the most output tokens (3,855 and 2,623 against kary's 1,576 and 1,323) because a $p-1$-deep fold pays a budget escalation at every level. Note that chain also shows the worst per-application latency spread of any algorithm: calibrating α from `sat-mb-fidelity__fidelity-chain-f-k4` gives a median of 47.5 s and a p99 of 219.1 s, against 24.0 and 25.0 s for the binomial run — the deep accumulator gets larger and slower to process as it descends.

The fidelity argument for avoiding chain is the one I would make most cautiously, and it is worth separating what is structural from what was measured. Structurally, chain re-encodes the accumulator $p-1$ times where a tree re-encodes it $\lceil \log_2 p \rceil$ times, and the re-encoding is demonstrably real: the sweep logs $p-1$ distinct content digests per run under `chain`, one new artifact per hop, where every broadcast algorithm logs exactly one digest no matter how many hops. That difference — a fold transforms, a copy does not — is why depth is a fidelity variable for a reduction and purely a latency variable for a broadcast, and it is the most defensible thing this family and `bcast/chain` establish jointly. But the repository's own fidelity experiment does *not* confirm that the transformation loses information as a function of depth: retention was 0.3854 under all four algorithms at $p = 8$ (`inc-mb-fidelity-inc`, 37 of 96 items surviving at fold depth 7, 3 and 2 alike), and 1.0000 under all four when the output budget was not binding (`sat-mb-fidelity`). Under a uniform enforced budget the last application is the binding one, so a 31-deep fold and a 5-deep fold discard the same amount — just at different places. So the case against chain reduction is cost, decisively; the fidelity case is a prediction the harness measured and did not confirm, and depth would govern quality only under a subtree-proportional budget that no archived run uses.

7 Threats to this reading

These runs contain no agents. All 21 report `executors = {"none": p}`, zero `agent.call` events, zero busy time, an idle fraction of 1.0 and $0.000 of cost, and the operator was `SUM` over a one-token scalar, so each of the $p - 1$ applications is an integer addition. The 80.2 ms per hop at $p = 32$ is an SQLite transaction plus an event append plus a thread wake-up; the cost model's α is on the order of 10^0–10^1 s, three orders of magnitude larger. So the measured chain-versus-binomial ratio of 24 is not the ratio an agent population would see, and I would not quote it as one — the model's 6.2 and the archived agent runs' 8 to 17 at $p = 8$ are the defensible figures.

Against that, this is one of only two families in my set whose wall times are not swamped by process launch. In `mb-free__coll-reduce-chain-32` the thirty-two `rank.init` events span 2.0 to 42.1 ms while the run lasts 2,494.3 ms, so start-up is under 2% of the total; the flat and binomial families at the same p have launch spreads of 61 and 91 ms against run walls of 76 and 104 ms, where essentially none of the elapsed time is the collective. That contrast is precisely why the chain families' durations can carry an argument and their siblings' cannot, and it is also a reason to be suspicious of any comparison that puts the two on the same axis.

The superlinear per-hop cost is an inference I have not isolated. The support is that nothing in the algorithm changes with p, that the same rise appears in `bcast/chain`, and that it does not appear where fewer ranks are blocked at once. A run with the same p and a shorter dependency chain, or a fabric instrumented for lock contention, would discriminate; neither exists in this archive and I have not re-run anything.

Several code paths are unexercised, and one of them is the interesting one. All 21 runs used `root = 0`, so the extra final hop that chain takes when the root is elsewhere never runs — and that hop makes the message count p rather than $p - 1$, which is a discrepancy the closed form $(p - 1)$ would report as a model disagreement. That is arguably a gap in the model rather than in the implementation, and no archived run tests it either way. All 21 used a commutative, EXACT operator, so the ordered left-operand discipline that is chain's entire reason for existing is validated here only by the counts agreeing, not by any order-sensitive result; the evidence for it is `test_reduce_preserves_order_for_noncommutative_op`. All 21 used a one-token payload, so the volume term is validated only at $n = 1$, and the envelope under-report above is measured only in the regime where the envelope dominates. And zero of 1,110 receptions across the six sweeps admitted a token into a rank's context, so chain's one genuine advantage over flat — a per-rank fan-in of one, and therefore the smallest context footprint of any reduce algorithm — is not exercised at all.